32-bit MIPS

Pipelined

Verification

Amir Jamshidi Bandari

Electrical Engineering

Toronto Metropolitan University

April 2026

PIPELINED VERIFICATION SUMMARY:

Unlike the single-cycle verification document, this document focuses on verifying **pipeline behavior**, **data hazards**, and **control hazards**. The main hazards tested include: forwarding, stalling, branching, jumping.

The goal of this verification is to confirm that the complete pipelined CPU works correctly when multiple hazards occur in the same instruction sequence.

Since pipelined signals are harder to debug manually, a self-checking testbench was added to speed up verification. The testbench uses conditional statements to compare actual CPU outputs against expected values, then prints whether each result is correct or incorrect.

The table below summarizes the pipeline and hazard verification process:

Module	Verify	Cause	Reverify
Forwarding Unit	3 Errors	1 Architectural 2 Edge Cases	Clear
Stalling Unit	2 Errors	1 Architectural 1 Architectural & Timing	Clear
Control Hazard Unit	Clear	-	-

Overall, the forwarding unit had one architectural issue and two edge-case issues.

The architectural issue was caused by comparing the wrong destination register field in the pipeline. The edge-case issues were caused by mixed instruction sequences where forwarding was either missing or incorrectly triggered.

During stalling unit verification, one architectural issue caused duplicate stalling because the same instruction remained active for extra clock cycles. Another issue was a read-after-write timing mismatch found during a chained dependency test.

Additional smaller issues were also found during pipeline verification. These issues were fixed and reverified. After the final full-pipeline and hazard tests, the CPU was confirmed to be ready for FPGA implementation.

Note: Most verification tests require setup instructions before the target behavior occurs. Because of this, some screenshots may show extra clock cycles before the tested signal behavior appears.

- Forwarding Unit

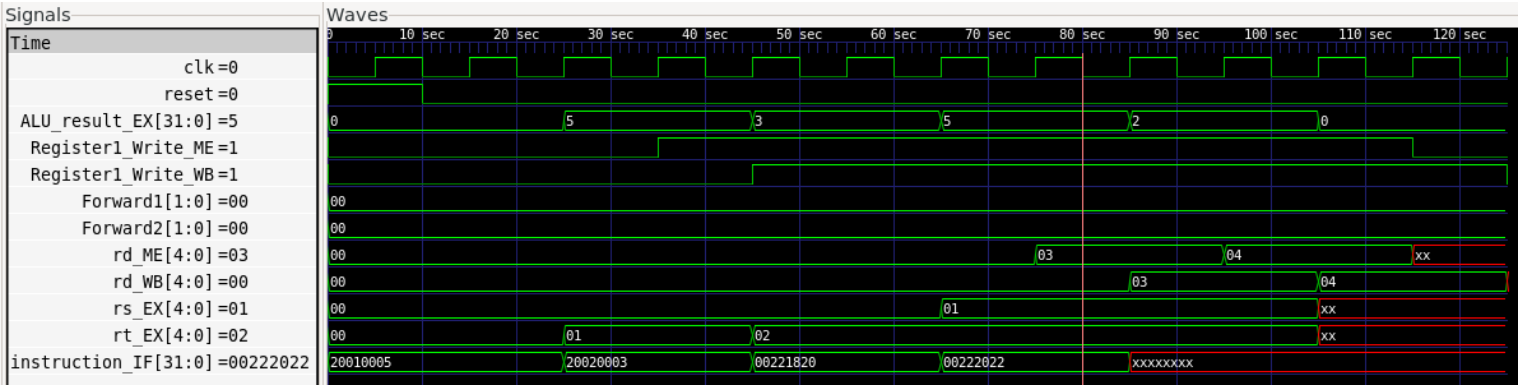


Image 1

Test Objective:

Verify that the forwarding unit correctly forwards the result of an earlier instruction to later dependent instructions.

This test uses four instructions. The result of the second instruction must be forwarded to the third and fourth instructions.

Signals Observed:

- Forward1 (binary)
- Forward2 (binary)

Expected Behavior:

Instruction	Register Destination (result)	Source 1 Register (value)	Source 2 Register/Immediate (value)
Addi	$1 (5)	$0 (0)	5
Addi	$2 (3)	$0 (0)	3
Add	$3 (8)	$1 (5)	$2 (3)
Sub	$4 (2)	$1 (5)	$2 (3)

Observed Behavior:

The first signals checked were Forward1 and Forward2, because these signals control whether operand forwarding occurs.

A value of 00 represents the default case, meaning no forwarding is selected. When a forwarding condition is detected, the signal should change to select the correct forwarded source, such as the memory stage or writeback stage.

However, the initial waveform showed that Forward1 and Forward2 were not selecting the correct forwarding paths. The self-checking testbench confirmed the issue:

The failure in R3 shows that the add instruction did not receive the correct forwarded value for one of its operands.

```
PASS R1 Value = 5
PASS R2 Value = 3
FAIL R3 Value = 5
PASS R4 Value = 2
```

Root Cause:

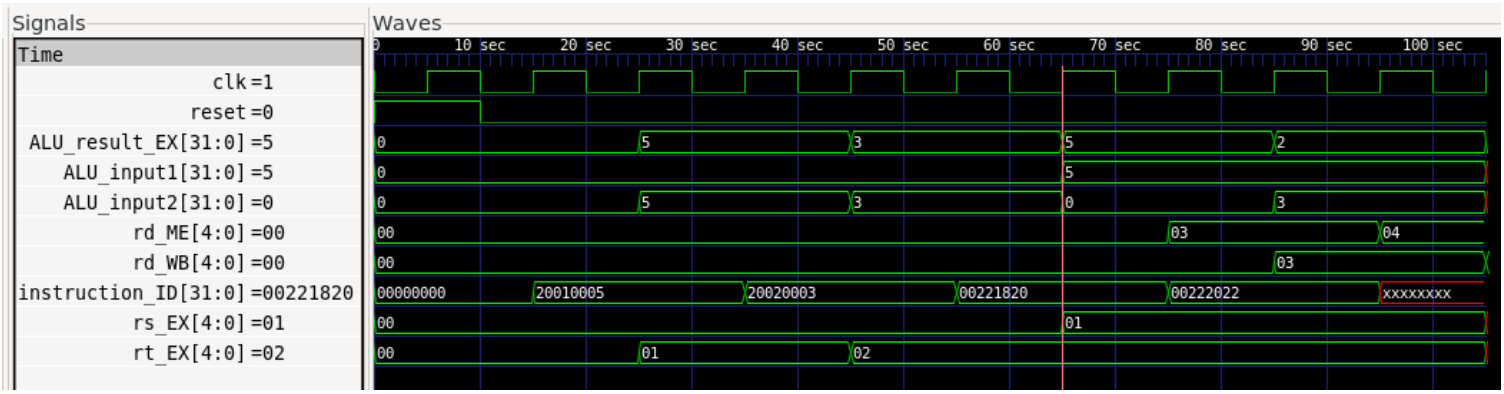


Image 2

At approximately 65 seconds, the source registers are correctly selected for the ADD instruction. The first source register, rs_EX, has the correct value of 5. However, the second source register, rt_EX, has the correct address but ALU_input2 is equal to 0.

This means the ALU is using the reset/default value instead of the forwarded value.

The forwarding unit is designed to compare the source registers of the current instruction with the destination registers of previous instructions. If the registers match, forwarding should occur.

In the original design, the forwarding unit compared:

rs_EX and rt_EX

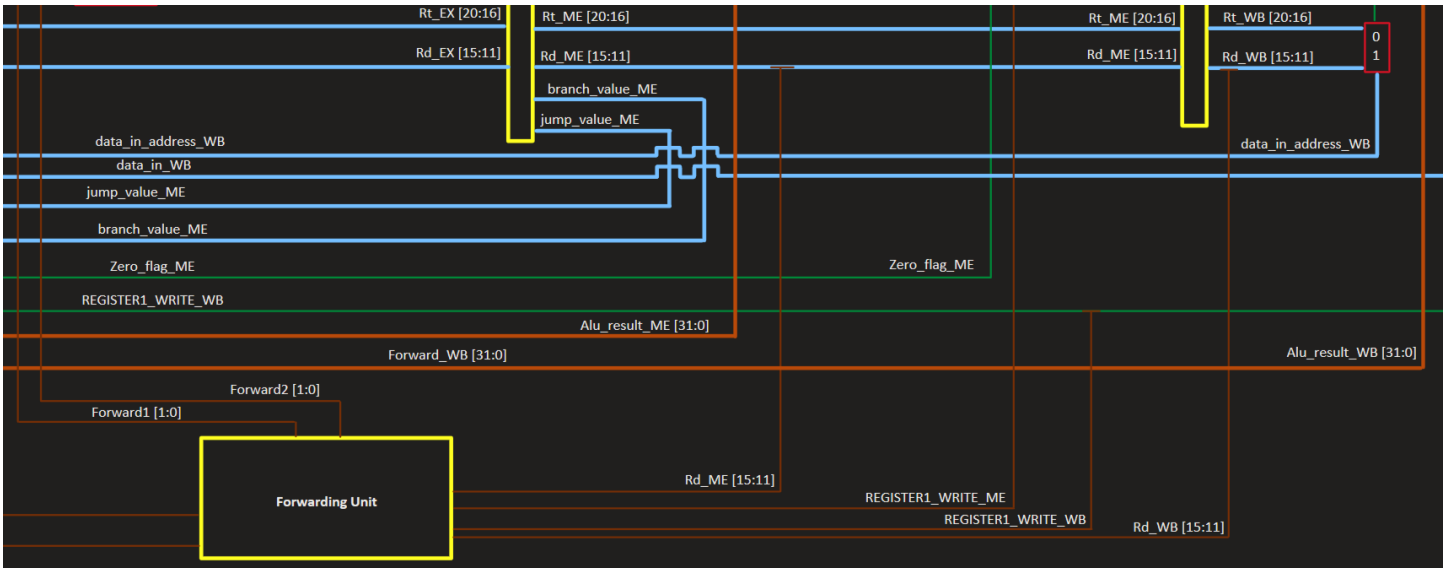
against:

rd_ME and rd_WB

This works for R-type instructions, where the destination register is rd. However, it fails for I-type instructions such as ADDI, because I-type instructions use rt as the destination register.

In this test, the value being forwarded comes from an ADDI instruction. Since the forwarding unit only checked rd_ME and rd_WB, it did not detect the correct destination register. As a result, forwarding did not occur correctly.

Debugging operation:



The design diagram showed that the forwarding unit was receiving rd_EX and rd_WB for destination comparison, while rt_EX and rt_WB were not being routed into the forwarding unit. This was the main design issue.

To fix this, the forwarding unit needed to compare the source registers against the actual writeback destination register, not only the rd field.

The better solution was to use data_in_address as the destination register address. This signal already represents the final selected destination register after choosing between rt and rd.

Before the fix, the forwarding unit only checked R-type destination fields:

Compare rs_EX / rt_EX with rd_ME / rd_WB

Before:

```
always_comb begin
    Forward1 = 2'b00;
    Forward2 = 2'b00;
    if ((Register1_Write_ME == 1) && (rd_ME != 0)) begin
        if (rd_ME == rs_EX)
            Forward1 = 2'b01;
        if (rd_ME == rt_EX)
            Forward2 = 2'b01;
    end
    if ((Register1_Write_WB == 1) && (rd_WB != 0)) begin
        if ((Forward1 == 2'b00) && (rd_WB == rs_EX))
            Forward1 = 2'b10;
        if ((Forward2 == 2'b00) && (rd_WB == rt_EX))
            Forward2 = 2'b10;
    end
end
```

After:

```
always_comb begin
    Forward1 = 2'b00;
    Forward2 = 2'b00;
    if ((Register1_Write_ME == 1) && (data_in_address_ME != 0)) begin
        if (data_in_address_ME == rs_EX)
            Forward1 = 2'b01;
        if (data_in_address_ME == rt_EX)
            Forward2 = 2'b01;
    end
    if ((Register1_Write_WB == 1) && (data_in_address_WB != 0)) begin
        if ((Forward1 == 2'b00) && (data_in_address_WB == rs_EX))
            Forward1 = 2'b10;
        if ((Forward2 == 2'b00) && (data_in_address_WB == rt_EX))
            Forward2 = 2'b10;
    end
end
```

After the fix, the forwarding unit checks the selected destination address:

Compare rs_EX / rt_EX with data_in_address_ME / data_in_address_WB

This allows forwarding to work for both R-type and I-type instructions.

Several smaller updates were also required to keep the pipeline consistent:

Move register destination selection earlier in the datapath.
Remove destination selection from the writeback-only path.
Add the selected destination address into the EX and ME pipeline registers.

New Observed Behavior:

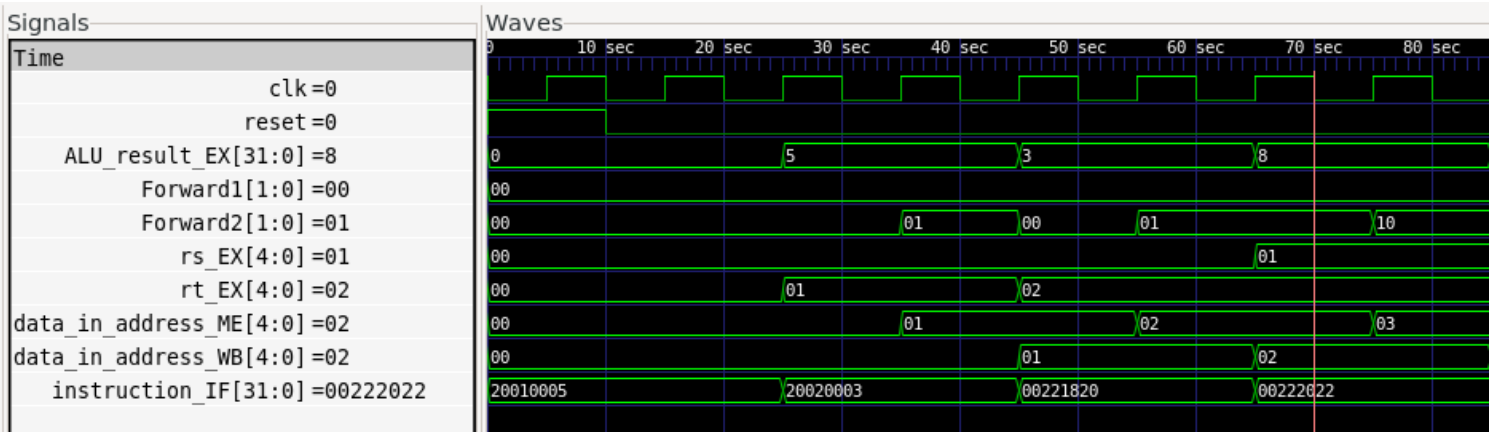


Image 3

After the update, the forwarding unit correctly selects forwarded values from the memory stage and writeback stage.

The ALU input MUXes also select the correct forwarded operands. In this case, the ALU receives the forwarded result and performs the arithmetic operation correctly.

The self-checking testbench now shows:

```
PASS R1 Value = 5
PASS R2 Value = 3
PASS R3 Value = 8
PASS R4 Value = 2
```

This confirms that forwarding now works correctly for the tested instruction sequence.

Conclusion:

The forwarding unit originally failed because it only handled R-type destination registers and did not properly support I-type instructions such as ADDI.

After updating the design to use the selected destination register address, forwarding works for both R-type and I-type instructions. The testbench results match the expected values, and no further issues were found in the forwarding unit.

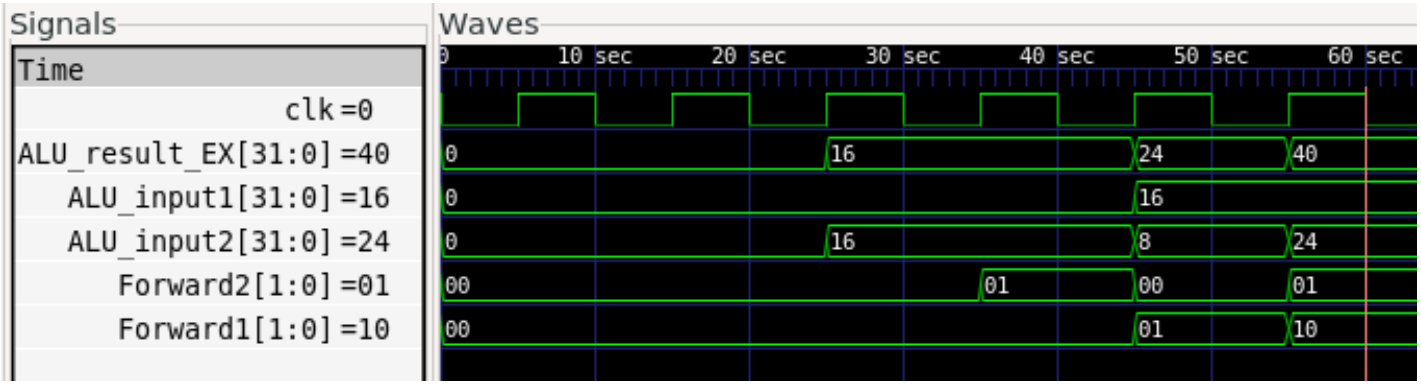


Image 4

Test Objective:

Further test the forwarding unit by mixing different instruction types and confirming that forwarding only occurs when it is actually needed.

Signals Observed:

- Forward1 (binary)
- Forward2 (binary)
- ALU_input1 (signed decimal)
- ALU_input2 (signed decimal)

Expected Behavior:

Instruction	Register Destination (result)	Source 1 Register (value)	Source 2 Register/immediate (value)
Addi	$1 (16)	$0 (0)	16
Addi	$3 (24)	$1 (16)	8

Observed Behavior:

Further testing showed that the second instruction produced 40 instead of the expected value 24 at 60 sec.

The self-checking testbench showed:

PASS R1
FAIL R3 = 40

Forward1 behaves correctly because the second instruction depends on the result of the first instruction. However, Forward2 misbehaves because it should not trigger during I-type instructions such as ADDI.

For an ADDI instruction, the second ALU input should come from the immediate value, not from a forwarded register value.

Root Cause:

The issue is caused by unnecessary forwarding on the second ALU input.

At 50 sec, the result of the first instruction is 16. The second instruction should calculate:

16 + 8 = 24

However, the forwarding unit incorrectly forwards 16 into ALU_input2, replacing the immediate value 8. As a result, the ALU calculates:

16 + 24 = 40

or effectively adds an incorrectly forwarded value instead of using the immediate.

At the same time, Forward2 = 01, which means forwarding is being selected for the second operand. During I-type instructions, this should not happen because the second ALU operand comes from the immediate field, not from rt.

The problem comes from the forwarding condition comparing data_in_address_ME with rt_EX. For I-type instructions, rt is the destination register, not the second source register. Therefore, matching rt_EX during an I-type instruction can falsely trigger forwarding.

Debugging operation:

The original forwarding logic allowed Forward2 to trigger whenever the selected destination register matched rt_EX.

However, this is only valid when the execute-stage instruction actually uses rt_EX as a source operand. For I-type instructions such as ADDI, the ALU uses an immediate value instead, so forwarding into the second operand should be blocked.

The fix was to add an ALUSrc_EX condition to the Forward2 logic. This prevents Forward2 from triggering when the instruction uses an immediate operand.

Before the fix:

Forward2 could trigger when destination == rt_EX

```
always_comb begin
    Forward1 = 2'b00;
    Forward2 = 2'b00;
    if ((Register1_Write_ME == 1) && (data_in_address_ME != 0)) begin
        if (data_in_address_ME == rs_EX)
            Forward1 = 2'b01;
        if ((data_in_address_ME == rt_EX))
            Forward2 = 2'b01;
    end
    if ((Register1_Write_WB == 1) && (data_in_address_WB != 0)) begin
        if ((Forward1 == 2'b00) && (data_in_address_WB == rs_EX))
            Forward1 = 2'b10;
        if ((Forward2 == 2'b00) && (data_in_address_WB == rt_EX))
            Forward2 = 2'b10;
    end
end
```

After the fix:

Forward2 only triggers when destination == rt_EX and ALUSrc_EX == 0

This ensures that forwarding into the second ALU input only happens for instructions that actually use register rt as the second source operand.

```
always_comb begin
    Forward1 = 2'b00;
    Forward2 = 2'b00;
    if ((Register1_Write_ME == 1) && (data_in_address_ME != 0)) begin
        if (data_in_address_ME == rs_EX)
            Forward1 = 2'b01;
        if ((data_in_address_ME == rt_EX) && (ALUSrc_EX == 0))
            Forward2 = 2'b01;
    end
    if ((Register1_Write_WB == 1) && (data_in_address_WB != 0) ) begin
        if ((Forward1 == 2'b00) && (data_in_address_WB == rs_EX))
            Forward1 = 2'b10;
        if ((Forward2 == 2'b00) && (data_in_address_WB == rt_EX) && (ALUSrc_EX == 0))
            Forward2 = 2'b10;
    end
end
```

New Observed Behavior:

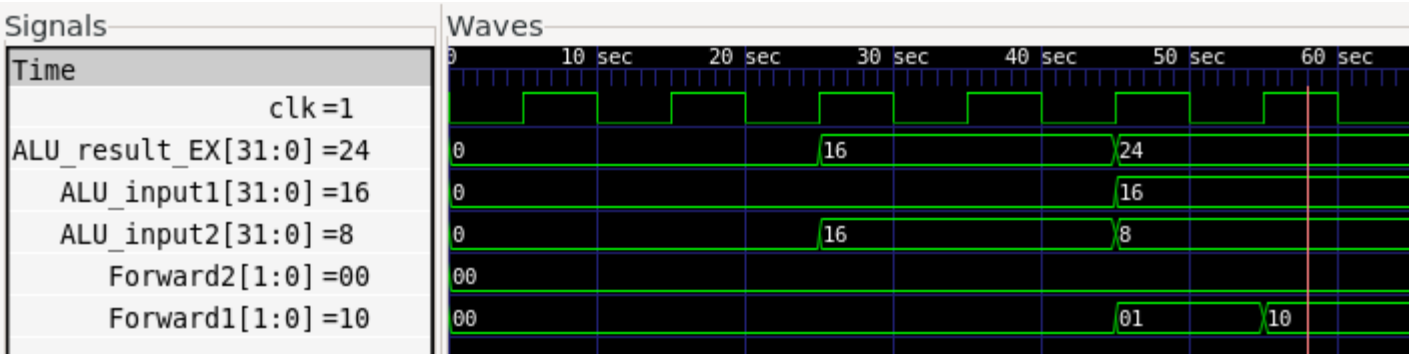


Image 5

After the update, Forward2 no longer sends the wrong value to the ALU during I-type instructions.

At 60 sec:

ALU_input1 = 16
ALU_input2 = 8
ALU_result = 24
Forward2 = 00

This matches the expected behavior. Forward1 still forwards correctly because the second instruction depends on the first instruction's result, while Forward2 remains at its default value because the second operand comes from the immediate field.

Conclusion:

The issue was caused by Forward2 incorrectly treating rt_EX as a source register during I-type instructions.

After restricting Forward2 so it only activates when the instruction actually uses rt as a source operand, the forwarding unit behaves correctly. The ALU now uses the immediate value for I-type instructions while still allowing valid forwarding on Forward1.

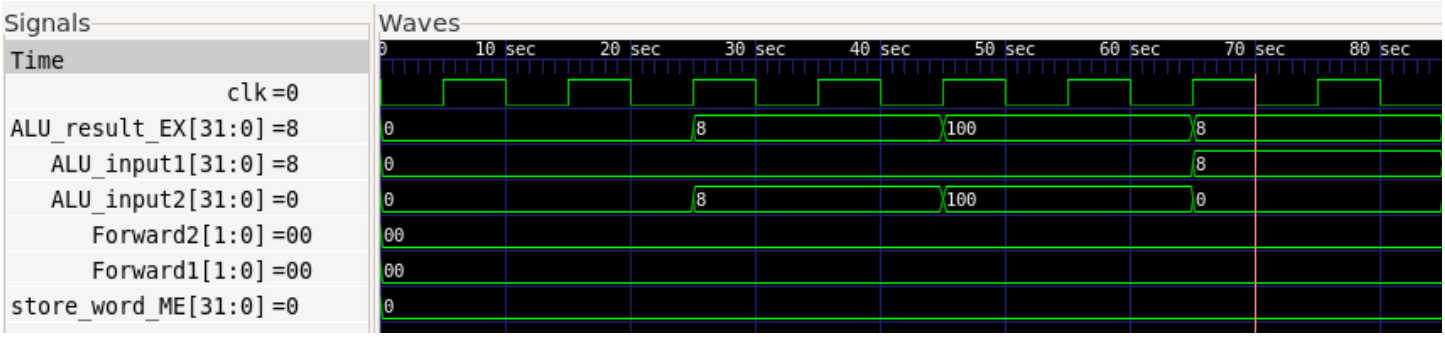


Image 6

Test Objective:

Further test the forwarding unit by mixing different instruction types and adding a store word instruction.

This test verifies that the forwarding unit can correctly forward a value into the store-data path when a sw instruction depends on a previous instruction.

Signals Observed:

- Forward2 (binary)
- ALU_result_EX (signed decimal)
- store_word_ME (signed decimal)

Expected Behavior:

Instruction	Register Destination (result)	Source 1 Register (value)	Source 2 Register/immediate (value)
Addi	$1 (8)	$0 (0)	8
Addi	$2 (100)	$0 (0)	100
sw	$2 (100)	$1 (8)	$0 (0)

Observed Behavior:

The store word instruction has two important parts:

1. The memory address
2. The data value being stored

The memory address is calculated using the base register and immediate offset. In this test, the address is calculated correctly:

R1 + 0 = 8

However, the store value is incorrect. The value that should be stored is $2 = 100, but store_word_ME is shown as 0.

The self-checking testbench confirms the issue:

PASS R1
PASS R2
FAIL MEM[2] = 0

This means the address calculation works, but the value being stored into memory is not being forwarded correctly.

The expected behavior requires forwarding the result of the second addi instruction into the store-data path of the sw instruction. However, during the sw instruction, Forward2 remains at its default value instead of selecting the forwarded value.

Root Cause:

The root cause is in the forwarding unit.

In the previous forwarding fix, Forward2 was restricted so it would not trigger during I-type instructions when the ALU uses an immediate value. This fixed the ADDI issue because ADDI should not forward into ALU_input2.

However, sw is also an I-type instruction, but it is different from ADDI.

For sw:

ALU_input2 uses the immediate offset for address calculation.
store_word uses rt as the data to write into memory.

So even though the ALU second input should not use forwarding, the store-data path still may need forwarding.

The previous restriction blocked Forward2 for I-type instructions, which accidentally prevented store-data forwarding for sw.

Debugging operation:

The previous forwarding logic was effectively saying:

If the current instruction is not R-type, do not trigger Forward2.

This works for instructions like ADDI, but it is too restrictive for sw.

The fix is to allow Forward2 when either:

1. The instruction uses rt as the second ALU source, meaning ALUSrc_EX == 0

or

2. The instruction is a store word instruction, meaning Memory1_Write_EX == 1

Before the fix:

Forward2 only allowed when ALUSrc_EX == 0

```
always_comb begin
    Forward1 = 2'b00;
    Forward2 = 2'b00;
    if ((Register1_Write_ME == 1) && (data_in_address_ME != 0)) begin
        if (data_in_address_ME == rs_EX)
            Forward1 = 2'b01;
        if ((data_in_address_ME == rt_EX) && (ALUSrc_EX == 0))
            Forward2 = 2'b01;
    end
    if ((Register1_Write_WB == 1) && (data_in_address_WB != 0) ) begin
        if ((Forward1 == 2'b00) && (data_in_address_WB == rs_EX))
            Forward1 = 2'b10;
        if ((Forward2 == 2'b00) && (data_in_address_WB == rt_EX) && (ALUSrc_EX == 0))
            Forward2 = 2'b10;
    end
end
```

After the fix:

Forward2 allowed when ALUSrc_EX == 0 OR Memory1_Write_EX == 1

This allows the forwarding unit to distinguish store instructions from other I-type instructions.

For ADDI, Forward2 remains blocked because the immediate value should be used.

For sw, Forward2 is allowed because the store-data value may need to be forwarded into the memory write path.

```
always_comb begin
    Forward1 = 2'b00;
    Forward2 = 2'b00;
    if ((Register1_Write_ME == 1) && (data_in_address_ME != 0)) begin
        if (data_in_address_ME == rs_EX)
            Forward1 = 2'b01;
        if ((data_in_address_ME == rt_EX) && (ALUSrc_EX == 0 || Memory1_Write_EX == 1))
            Forward2 = 2'b01;
    end
    if ((Register1_Write_WB == 1) && (data_in_address_WB != 0) ) begin
        if ((Forward1 == 2'b00) && (data_in_address_WB == rs_EX))
            Forward1 = 2'b10;
        if ((Forward2 == 2'b00) && (data_in_address_WB == rt_EX) && (ALUSrc_EX == 0 || Memory1_Write_EX == 1))
            Forward2 = 2'b10;
    end
end
```

New Osbserved Behavior:

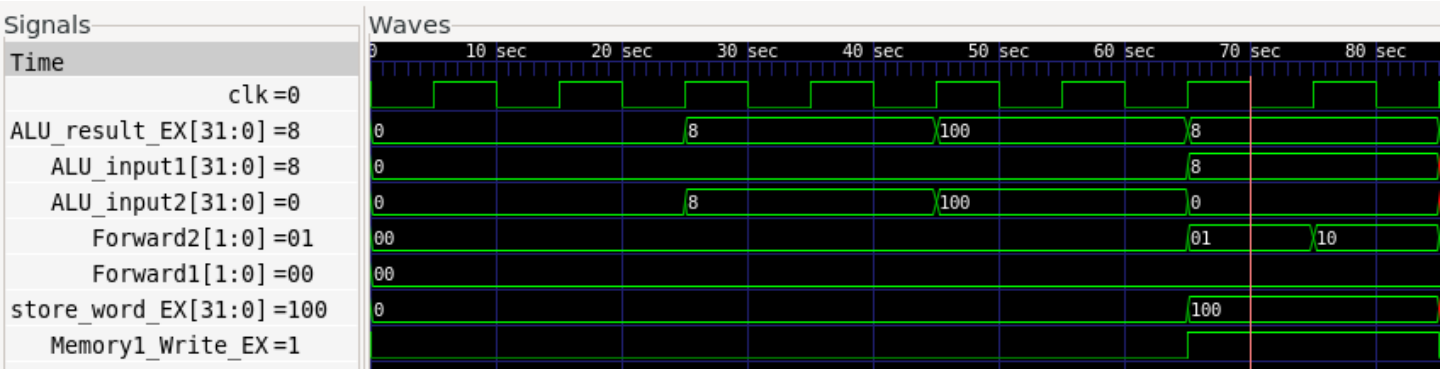


Image 7

After the update, Forward2 correctly becomes non-default while the memory write signal is high.

The forwarded result from the previous instruction is sent to the MUX connected to the store-data path. The waveform shows:

store_word_EX = 100
Memory1_Write_EX = 1

This confirms that the store word instruction now receives the correct value to write into memory.

Conclusion:

The issue was caused by the previous forwarding restriction, which fixed ADDI but accidentally blocked forwarding for sw.

After updating the forwarding condition to allow Forward2 when Memory1_Write_EX is high, the forwarding unit correctly supports store word instructions without breaking the previous I-type forwarding fix.

The forwarding unit can now distinguish between:

ADDI: use immediate, do not forward into ALU_input2
SW: use immediate for address, but allow forwarding for store data

- Stalling Unit

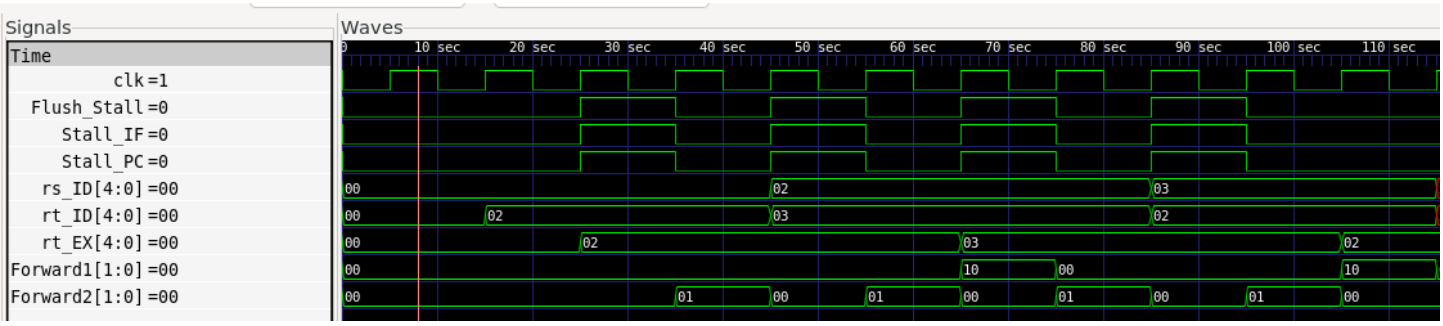


Image 8

Test Objective:

Verify that the stalling unit correctly triggers when a load-use hazard occurs.

A load-use hazard happens when the destination register of a load instruction in the execute stage matches one of the source registers of the following instruction in the decode stage.

Signals Observed:

- rs_ID (hex)
- rt_ID (hex)
- rt_EX (hex)
- Stall_IF
- Stall_PC
- Flush_Stall

Expected Behavior:

Match At time	rt_EX == rs_ID 50 sec
Stall_IF	1
Stall_PC	1
Flush_Stall	1

Observed Behavior:

The waveform generated at 50 sec and 90 sec matches the expected stall condition. However, the stall occurs twice when it should only occur once.

At 50 sec, the first stall is correct because the load instruction's destination register matches the source register of the following instruction. However, the same instruction appears to exist in two different pipeline stages at the same time instead of being replaced by the next instruction. This causes the stalling unit to detect the same hazard again later.

Root Cause:

Previous waveform checks showed that instructions appeared to stay active for two clock cycles. This suggested an issue with the program counter update path.

The program counter was not receiving the newest PC value quickly enough. Because of this, the instruction memory could output the same instruction for more than one cycle. That repeated instruction was then stored into the IF/ID pipeline register and passed to the instruction decoder again.

The original program flow was effectively:

PC → PC Adder → IF Pipeline Register → PC Writeback → New PC

With respect to the rising clock edge, this caused the PC update to depend on a value stored in the IF pipeline register. As a result, the program counter held an older value for an extra cycle, causing duplicate instructions.

The intended flow should be closer to:

PC → PC Adder → PC Writeback → New PC

This allows the PC writeback logic to use the current pc_adder_value directly, instead of waiting for the delayed pc_adder_IF value.

Debugging operation:

To fix the issue, the PC_Writeback.sv module was updated to use PC_Adder_value directly instead of PC_Adder_value_IF.

This means the PC no longer waits an extra clock cycle for the PC+4 value to come from the IF pipeline register.

This update prevents the PC from repeatedly fetching the same instruction under normal sequential execution.

Before:

```
always_comb
begin
    next_pc = pc_adder_IF;
    if (Jump_EX)
        next_pc = jump_value_EX;
    else if (Zero_Flag_EX && Branch_EX)
        next_pc = branch_adder_value_EX;
    else next_pc = pc_adder_IF;
end
```

After:

```
always_comb
begin
    next_pc = pc_adder_value;
    if (Jump_EX)
        next_pc = jump_value_EX;
    else if (Zero_Flag_EX && Branch_EX)
        next_pc = branch_adder_value_EX;
    else next_pc = pc_adder_value;
end
```

New Observed Behavior:

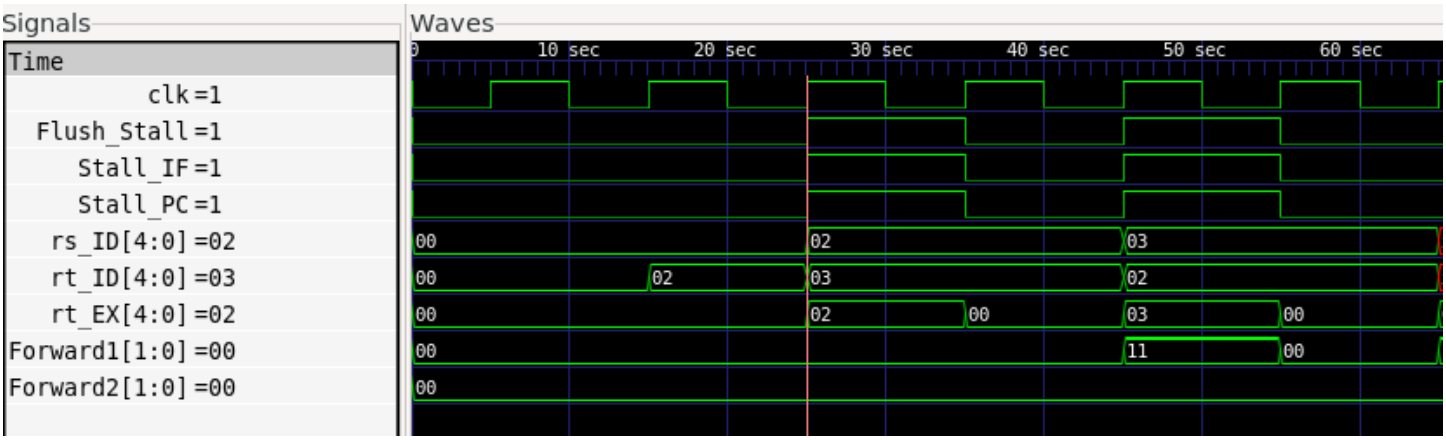


Image 9

After the fix, the program counter no longer duplicates instructions. The stalling unit now triggers once during the load-use hazard instead of triggering twice for the same instruction.

The updated waveform shows that when the hazard occurs, the correct stall signals are asserted:

Stall_IF = 1
Stall_PC = 1
Flush_Stall = 1

Then the pipeline continues normally after the stall is inserted.

The same test also completes faster after the fix. In the previous waveform, the sequence took about 115 sec, while after the PC update fix, the sequence takes about 65 sec. This confirms that duplicate instruction fetching was slowing down the CPU.

Conclusion:

The stalling unit itself was detecting hazards correctly, but the program counter update path caused duplicate instructions, which made the stall unit trigger more than once for the same hazard.

After updating the PC writeback logic to use the current pc_adder_value, duplicate instruction fetching was removed. The stall unit now works correctly with the forwarding unit and properly handles load-use hazards without unnecessary repeated stalls.

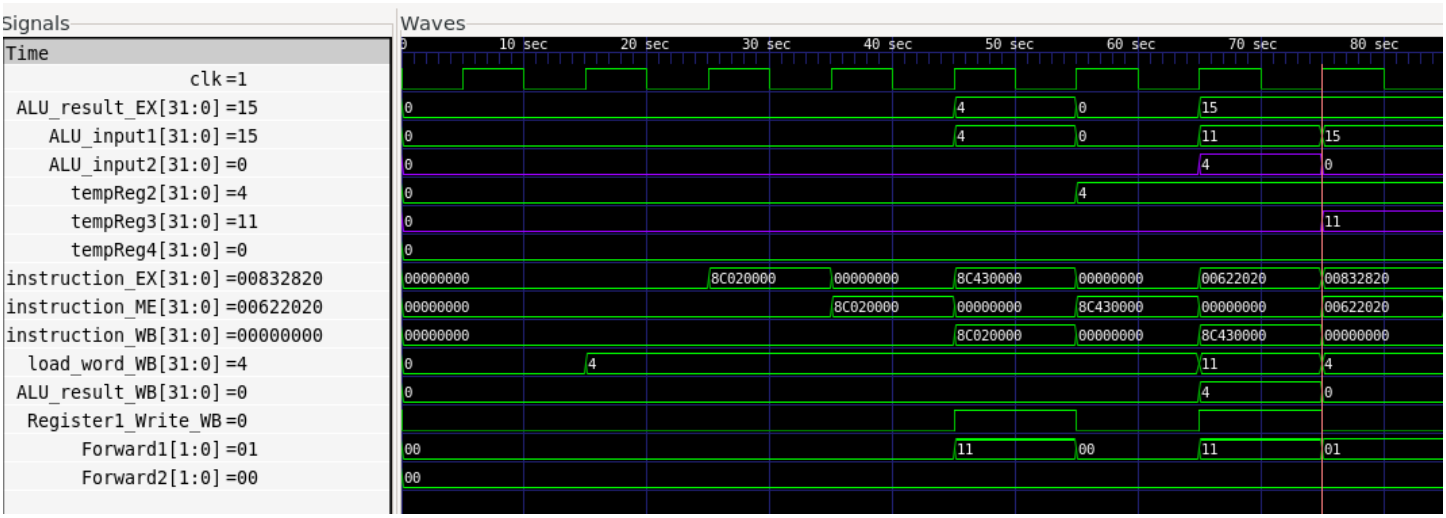


Image 10

Test Objective:

Test the stalling unit and forwarding unit together using a mixed instruction sequence with chained dependencies.

This test verifies that the processor can correctly handle a case where a later instruction depends on both:

1. A previous arithmetic result
2. A value loaded from memory

Signals Observed:

- ALU_result_EX (signed decimal)
- instruction_EX (hex)
- ALU_input1 (signed decimal)
- ALU_input2 (signed decimal)

Expected Behavior:

At time	70 sec	80 sec
instruction_EX	00622020	00832820
ALU_result_EX	15	26
ALU_input1	11	15
ALU_input2	4	11

Observed Behavior:

While testing forwarding and stalling together, the design exposed an issue in a chained dependency sequence.

The final instruction, 00832820, depends on:

1. The previous instruction, 00622020
2. The loaded value from instruction 8C430000

The issue is that ALU_input2 is not reading the correct loaded value. Instead of reading 11, it reads 0.

This causes the final ALU result to be incorrect.

Root Cause:

Temporary register outputs were added for debugging. At the rising clock edge, tempReg3 eventually becomes 11, but ALU_input2 reads 0 before the updated value is available.

This indicates a read-after-write timing mismatch in the register file.

The writeback value, load_word_WB, becomes 11 around the same time that the decode stage is trying to read that register. However, because register writes occur on the clock edge, the decode-stage read still sees the old value before the new value is fully available.

In other words:

Register writeback is updating the register.
Decode stage is reading the same register too early.

So the register file read receives the old value instead of the newly written value.

Debugging operation:

There are two common ways to solve this type of read-after-write issue:

1. Stall the pipeline for an extra cycle
2. Forward the writeback value directly into the decode-stage register outputs

Forwarding is the better solution here because it avoids adding unnecessary extra stalls.

The fix is to add a writeback-to-decode forwarding path inside the register file read logic. When the writeback stage is writing to the same register that the decode stage is reading, the register file output should use the writeback value directly instead of the old stored register value.

Before the fix, the register file read logic simply output the stored register values:

```
always_comb begin
    data_out_1_ID = register1[rs_ID];
    data_out_2_ID = register1[rt_ID];
end
```

After the fix, the register file checks whether the writeback destination matches either source register:

```
always_comb begin
    if ((Register1_Write_WB) && (data_in_address_WB != 0) && (data_in_address_WB == rs_ID))
        data_out_1_ID = data_in_WB;
    else
        data_out_1_ID = register1[rs_ID];

    if ((Register1_Write_WB) && (data_in_address_WB != 0) && (data_in_address_WB == rt_ID))
        data_out_2_ID = data_in_WB;
    else
        data_out_2_ID = register1[rt_ID];
end
```

This bypasses the timing mismatch by sending the newest writeback value directly to the decode-stage outputs.

New Observed Behavior:

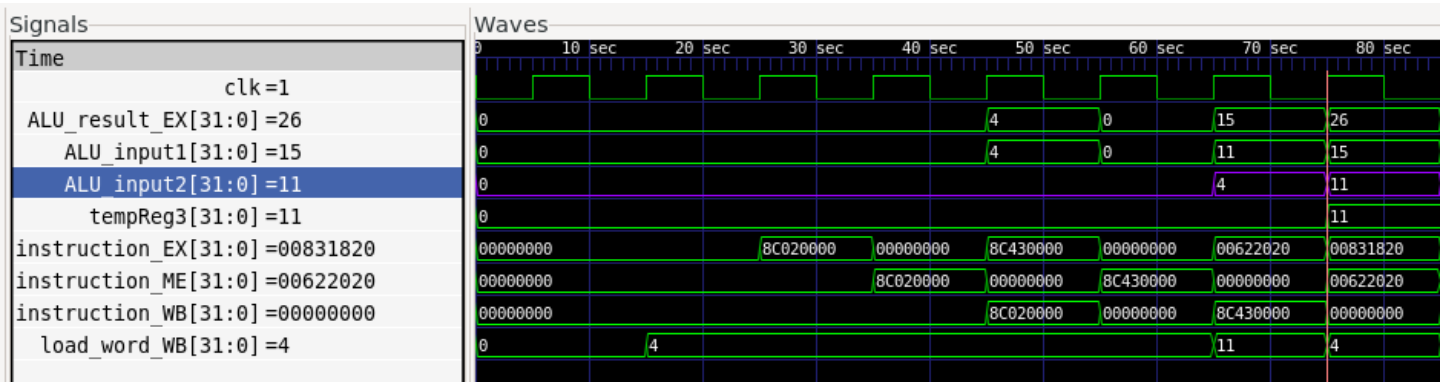


Image 11

After adding writeback-to-decode forwarding, the read-after-write issue is fixed.

The waveform now shows that ALU_input2 receives the correct value:

ALU_input2 = 11

The final instruction now calculates the correct result:

15 + 11 = 26

This matches the expected behavior.

Additional testing confirmed that the update does not break normal register reads or writeback behavior.

Conclusion:

This test exposed a timing mismatch between register writeback and register read. The decode stage was reading an old register value while the writeback stage was updating that same register.

The issue was fixed by adding writeback-to-decode forwarding. This allows the register file output to use the newest writeback value immediately when the decode-stage source register matches the writeback destination.

After this update, the stalling and forwarding units work correctly together during chained dependencies.

- Control Hazard Unit

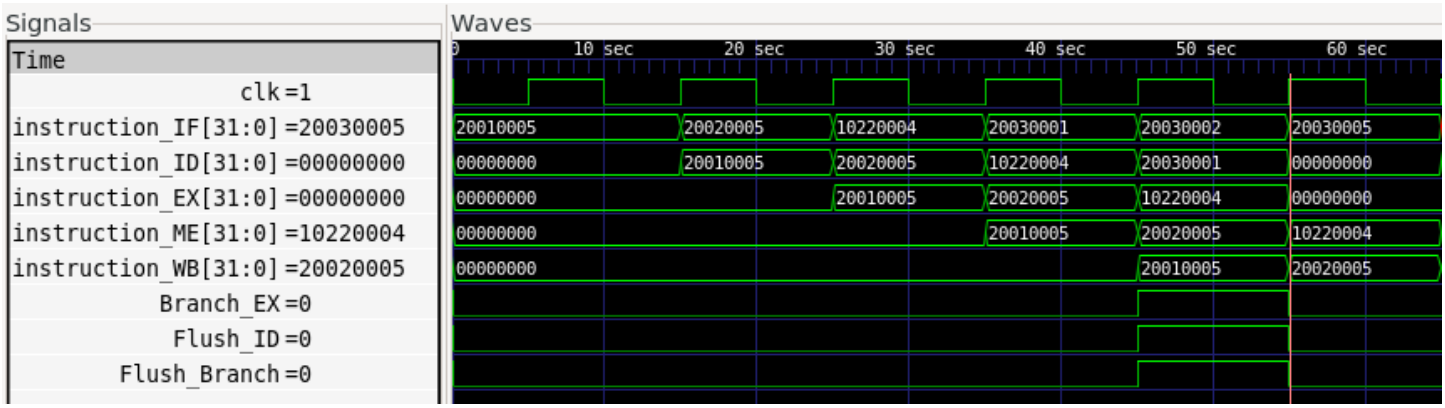


Image 12

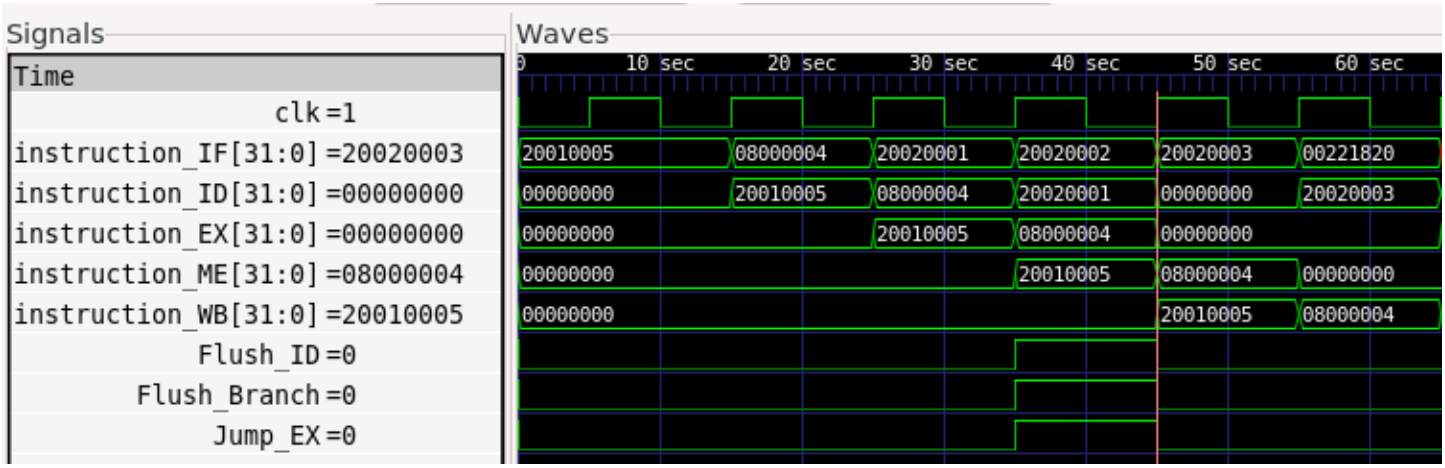


Image 13

Test Objective:

Verify that branch and jump instructions correctly flush the next two instructions and redirect the program counter to the correct target instruction.

Signals Observed:

- instruction_IF (hex)
- instruction_ID (hex)
- instruction_EX (hex)
- instruction_ME (hex)
- Jump_EX
- Branch_EX

Expected Behavior:

Tested Instructions for branching (image 12):

0. 20010005 → addi $1, $0, 5 → 0 + 5
1. 20020005 → addi $2, $0, 5 → 0 + 5
2. 10220004 → beq $1, $2, 4 → skip next 4 instructions
3. 20030001 → should be flushed
4. 20030002 → should be flushed
5. 20030003 → should be skipped
6. 20030004 → should be skipped
7. 20030005 → addi $3, $0, 5 → 0 + 5 → should execute

At time	55 sec
instruction_IF	20030005
instruction_ID	00000000
instruction_EX	00000000
instruction_ME	10220004

Tested Instructions for jumping (image 13):

20010005 → addi $1, $0, 5 → 0 + 5
08000004 → jump to instruction index 4
20020001 → should be flushed
20020002 → should be flushed
20020003 → should execute
00221820 → add $3, $1, $2 → 5 + 3

At time	45 sec
instruction_IF	20030003
instruction_ID	00000000
instruction_EX	00000000
instruction_ME	08000004

Observed Behavior:

At approximately 45 sec, the branch instruction is triggered, causing the flush signals to go high. At approximately 55 sec, the flush takes effect by inserting bubbles into the IF and ID pipeline registers.

The flushed instructions appear as:

00000000

This prevents incorrect instructions from continuing through the pipeline after a branch is taken.

The branch instruction also correctly redirects the program counter. Instead of fetching and executing the skipped instructions, the processor fetches the correct branch target instruction.

The jump instruction behaves similarly. Once the program counter jumps to the target instruction, the two incorrect instructions already inside the pipeline are flushed. The processor then selects the correct jump target instruction.

Conclusion:

The control hazard unit works correctly for both branch and jump instructions.

The unit successfully flushes the IF and ID pipeline registers after a control-flow change, preventing incorrect instructions from executing. The program counter is also redirected correctly so that the proper branch or jump target instruction is fetched. No issues were found in this unit.